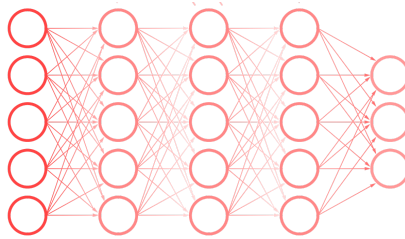


DEEP LEARNING FINAL PROJECT

REPORT



Team Sigma

Pol García López - 268414

Marc Rodríguez Vitolo - 268926

Clara Serra Borràs - 266503

Academic Year 2025/2026

Course: Deep Learning

Universitat Pompeu Fabra

Date: 08/06/2026

1. INTRODUCTION AND MOTIVATION

Not long ago, in the community of one of the members of the group, a group of people that was not from the building, jumped the fence and used the swimming pool. The neighbors were discussing what could be done to prevent this type of incident from happening again. The conventional proposal was to install a security camera and hire personnel to monitor the footage continuously. However, this approach has a critical drawback, it is very costly. That was where the idea emerged: an automated detection system that alerts residents only when a person is detected would be a far more cost-effective and reliable alternative.

The pool area is not illuminated at night, so standard RGB cameras are ineffective. Infrared cameras, however, capture heat radiation emitted by living bodies, making them a good choice for nighttime surveillance. The task is therefore framed as a binary image classification problem: given an infrared or low-light nighttime image, determine whether a person is present (label 1) or not (label 0), and automatically notify the community if the former is detected.

The main challenge is that the model must perform well under low-quality, nighttime conditions, with small, low-contrast human figures, variable backgrounds, and potential domain mismatch between training data (diverse thermal images from public datasets) and deployment data (pool-specific night images).

2. RELATED WORK / STATE OF THE ART

At the moment, the industry is now focusing more on developing detection models more than classification models. These detection models are able to classify but also to localize different types of objects. They are not only used in surveillance but also in autonomous vehicles, search and rescue applications and more. The ones that are more relevant right now and served as base for the designs made are YOLO and RTMDet.

2.1 YOLO

YOLO (You Only Look Once) is one of the most used architectures for real-time object detection. YOLO is able to perform localization and classification in a single forward pass through a convolutional backbone. This design makes it very efficient. The model that was taken as reference and used in this project was YOLOv8, developed and trained by Ultralytics. There is not an official paper for this model but all the documentation can be found on the Ultralytics webpage [1]. Below the architecture of YOLOv8 will be explained, as it will be important for later.

YOLOv8 is divided into 3 main parts [2]: the backbone, that is responsible for extracting the features of the images; the neck, that combines the features extracted from the backbone at different stages; and finally the head that is responsible for generating the output, both the classification of the object as the bounding box.

In Image 1 is shown the structure of the backbone of YOLOv8. It is composed of several convolutional layers of kernel 3x3, with stride = 2 and padding =1; combined with Concatenate to feature (C2f) blocks and a final Spatial Pyramid Pooling Layer (SPPF). Above each layer and block we can see the size of the data at that point. The convolutional layers make the image smaller while producing more channels to extract the features of the image. The C2f blocks, on the other hand, don't change the dimensions of the data and work in a completely different way. Inside the blocks the data first goes through a convolutional layer with kernel size 1x1 and stride and padding equal to 1. Then the channels are splitted in half: the first half goes through a sequence of "bottlenecks" and the second

half goes directly to the end. These bottlenecks are pairs of convolutional layers but they don't modify the size. At the end the first half and the different outputs of all the bottlenecks are concatenated and passed through a final convolutional layer with kernel 1×1 that reduces the number of channels back to its original value. The C2f block can be seen in Image 2.

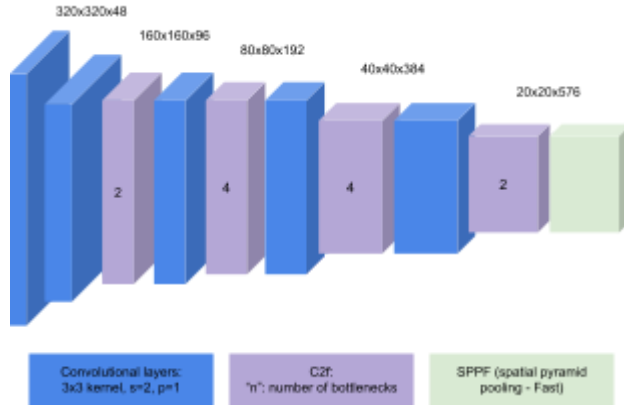


Image 1: YOLOv8 Backbone architecture

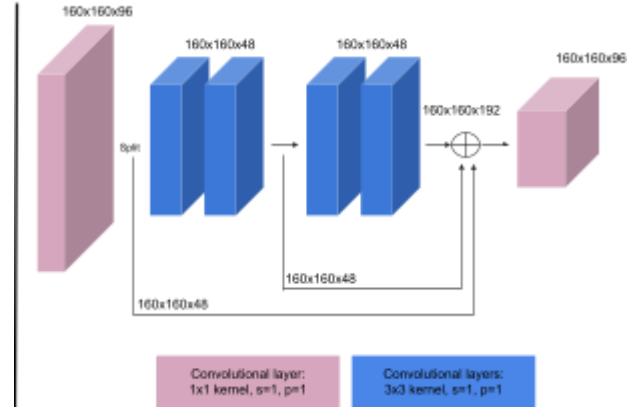


Image 2: Concatenate to feature (C2f) blocks design

2.2 RTMDet

Real-Time Models for Detection is a recent architecture developed by Alibaba [3]. It is designed with the objective of maximizing the tradeoff between quick performance and accuracy, maintaining a simple architecture. It employs convolutional layers with large kernels and a decoupled head. It has good results in benchmarks like COCO. Although in this project RTMDet was not implemented or used, it illustrated the importance of not only having a good performing model but also an efficient one, that can be used in real-time as in the use case treated here.

3. DATASET AND PREPROCESSING

In the development of this project 2 different datasets were used:

3.1 LLVIP Dataset

The Low-Light Visible and Infrared Paired Dataset is a public benchmark collected for pedestrian detection in night and low visibility conditions. It contains 15488 images and both the RGB and infrared version are provided. The images come from sequences captured from fixed surveillance cameras placed in the streets. For our project, only the infrared version was used.

Because this dataset is for detection purposes, all the images in the dataset contain people. This is a critical issue for us, because a classification model trained with this dataset would learn to always say that the image contains people. To solve this problem, the images were divided into 4 equal parts, but only under certain conditions. To keep things simple and don't have tricky images where only a small part of someone appears, only the images where all the bounding boxes lie inside only one of the parts are divided. While processing the splits the labels were also converted from bounding boxes to binary: for full images the label is 1 as it always contains people; for the parts, the algorithm checks if there is any bounding box inside that part and outputs 0 or 1 depending on this. As a result of this preprocessing 7001 images were divided and 8487 were not, that all together ended up making a dataset of 43492 images. From this total 25235 (58%) contained people and 18257 (42%) were only background.

This solution has advantages and disadvantages. The advantages are that it solved the problem of not having images without people in them and also that it provided images of different scales and more

variety. The main disadvantage is that the images come from fixed cameras, making duplicates of background images.

3.2 My Dataset (our own custom data)

To evaluate our model with a real use case, the group collected a custom dataset from a pool like the one that motivated the project. This dataset consists of 1333 images that were done with a standard security camera at night overlooking the pool from different angles and situations. The idea is to see how well the model performs with photographs taken with conventional cameras (non-infrared), that was the one the group member had available at the moment.

The images are 320x240 pixels, all in grayscale. Labels were manually assigned to the photographs taken, 978 (73.4%) images contain at least one person and 355 (26.6%) images contain nobody. The class imbalance is due to the fact that empty images become very similar to others and it's hard to get a variety of images with nobody in them. To have a more complete dataset, there are images that contain dogs (with and without people). The initial idea of this dataset is to be used as an evaluation of how the model could perform when the domain and type of image changes and see if it adapted well to all cases.

3.3 Data analysis

The data analysis revealed meaningful insights about both datasets. For the LLVIP, the images that contain people tend to have higher mean pixel intensity (76.7) compared to the images with no people in them (72.9), because in infrared images people appear as bright regions.



Image 3: Pixel intensity for class (LLVIP)

Image 4: Mean and standard deviation per image and boxplots (LLVIP)

In Images 3 and 4 the difference between both classes can be observed. In almost everything both classes overlap, but although it is not much, the intensity of background images have less intensity than images with people, and the mean and standard deviation have higher values in the images with people in them.

For My Dataset the statistics and results that were obtained were not equivalent. In this case the differences between both classes are reversed and the intensity is much darker. The mean of intensity for images with people is 35.2 while the mean for images without people is 41.9. This is also reflected in the different plots made during the analysis and that are shown in the Image 5 and 6. Here the metrics overlap even more as it can be seen in the boxplots and histograms.

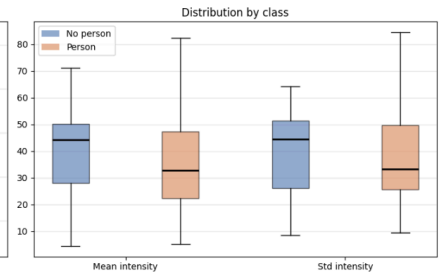
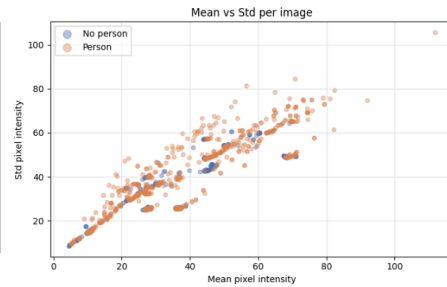
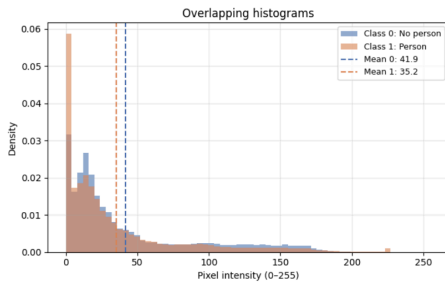


Image 5: Pixel intensity for class (My Dataset) Image 6: Mean standard deviation per image and boxplots (My Dataset)

3.4 Preprocessing of the data

All images were processed through a standardized preprocessing that was applied to both dataset and to all the splits:

- Resizing the images to 224x224 pixels to match the input size of the models
- Convert the images from RGB to a single-channel grayscale
- Convert the images into PyTorch tensors
- Normalize pixel values from the range $[0,1]$ to $[-1,1]$, centering around 0 for faster convergence

4. METHODOLOGY AND IMPLEMENTED MODELS

4.1 Baseline Model

The baseline model is a custom CNN designed from scratch and inspired by VGG. The architecture of the model can be seen in Image 7. The total number of parameters is approximately 6422785. This model uses Binary Cross Entropy as loss function and Stochastic Gradient Descent with momentum as optimizer. Additionally to the preprocessing done to the data, some simple data augmentation is performed so the model can generalize better: random horizontal flip, random rotation ($\pm 10^\circ$), random brightness and contrast adjustment.

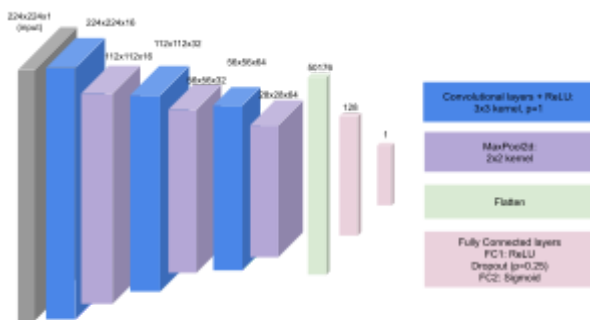


Image 7: Baseline model architecture

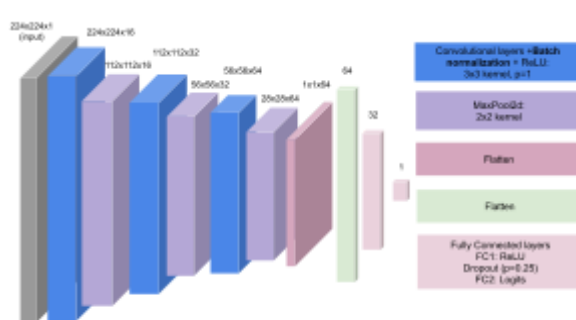


Image 8: Improved CNN architecture

4.2 Improved CNN

Based on the experiments described in Section 5, the final architecture incorporates several improvements over the baseline:

- Batch Normalisation layers added after each convolutional layer, before ReLU activation.
- Adaptive Average Pooling (1×1) replaces the flat MaxPool cascade, reducing the feature vector to 64 dimensions and dramatically cutting parameters.
- AdamW optimizer with weight decay, replacing SGD with momentum.

- BCEWithLogitsLoss replaces BCELoss, the sigmoid is removed from the final layer and the output are raw logits, then the loss function applies it internally with improved stability.
- Expanded data augmentation: adds Random Affine Transformation, Gaussian Blur, and Gaussian Noise to improve robustness to camera artifacts and motion blur.
- Reduced rotation range to $\pm 5^\circ$ (from $\pm 10^\circ$) based on domain knowledge about fixed surveillance cameras.

The total number of parameters is significantly under 100000, enabling faster training and forward pass (needed for real-time applications) and also less risk of overfitting. The architecture design can be seen in Image 8.

4.3 Transfer learning with YOLOv8

Once the previous models were evaluated in our dataset and showed a really bad performance, a completely new approach was tried: Transfer learning and use our own dataset to train.

The chosen model is YOLOv8, as explained in the State of the art, it is a detection model that is able to detect different classes of objects. The idea is to take the backbone of YOLO and take advantage of the feature extraction that it produces. It is interesting to us that it is able to detect all types of objects and not only people, as we don't want models that confuse other living or moving objects with humans. The final architecture of the model is shown in Image 9.

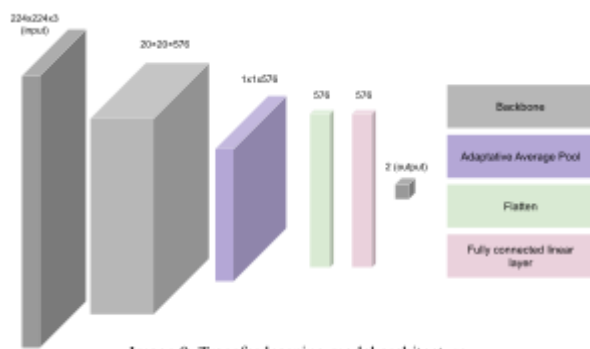


Image 9: Transfer learning model architecture

The model is composed by: the backbone of YOLOv8, which is already trained and the weights are frozen so they don't change during training; an adaptive average pool that takes the pixels of each channel and averages them to only have one value for channel; a flatten layer is then used to reduce the dimensions and finally a fully connected linear layer that uses the different channels that were produced to generate 2 logits.

The configuration for training this network is slightly different than the ones used before. Here Cross Entropy was used as a loss function as the network is designed to be used for a custom number of classes (here it's only done with 2), and uses Adam as optimizer. The images are normalized to mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]). [4]

5. EXPERIMENTS AND EVALUATED CONFIGURATIONS

5.1 Baseline model training process

The baseline model was trained with the LLVIP dataset for 10 epochs, with the configuration explained before. The dataset was already splitted in test and train, and we split the train part in 80% for training and 20% for validation.

5.2 From the baseline model to our best one

From the baseline, which performed very well (see part 6.1), emerged one main objective. The baseline model contained approximately 6.5 million parameters. This big model took long both to train and to do forward passes to evaluate images. Given that it is planned to be integrated in a simple surveillance system and has to work in real-time, the efficiency of the model has to be as good as the performance. From here several incremental updates were done to the baseline model to try reducing the size while maintaining the performance. The different changes are the following:

- First an Adaptive Average Pooling layer was added before the flattening so the input that goes into the fully connected layer is much smaller than it is in the baseline (from 50176 to 64). The number of total parameters of the model drops from 6.5M to less than 100.000. This reduction also helps avoiding overfitting and getting poor generalization. The accuracy and f1 score is expected to be lower than the baseline.
- Change the optimizer from SGD with momentum to AdamW. The idea is to evaluate if changing the optimizer helps getting faster and better convergence. The decay makes the last iterations advance less and get closer to the minimum.
- Change the loss function from Binary Cross Entropy to Binary Cross Entropy with Logits. This second loss function works directly with raw logits. It combines the sigmoid and BCE in a single numerically stable operation. Here the sigmoid that was done at the last layer was also deleted as it is automatically applied in the loss function.
- The data augmentation pipeline was extended to play with Gaussian Blur, Gaussian Noise, Random Affine transformations, and reduced rotation range ($\pm 5^\circ$ instead of $\pm 10^\circ$, appropriate for fixed surveillance cameras).
- Batch normalization was added to stabilize the distribution of activations during training and accelerate the convergence. BatchNorm was added after each convolutional layer, before doing the activation.

5.3 Transfer Learning

The model was trained only using the images in My Dataset. The data was splitted into 3 parts, 70% of the images were used for training, 15% for validation and the remaining 15% for testing. In this case, due to the small number of parameters that need to be trained and the small amount of data, the model was trained for 5 epochs.

6. RESULTS AND ANALYSIS

6.1 Baseline CNN model

The baseline model performed very well. After the 10 epochs the final train accuracy was 96% and f1 score of 0.96 while the final validation accuracy was 97% and f1 score of 0.97. The validation metrics are optimistic in this case, as training and validation splits can contain very similar images (produced by the same security camera). The model converged smoothly as we can see in Image 10, with both curves following closely. From the plot in Image 11 it can be seen that the model correctly separates both classes completely from the validation set, in this case moving the threshold wouldn't change much the results.

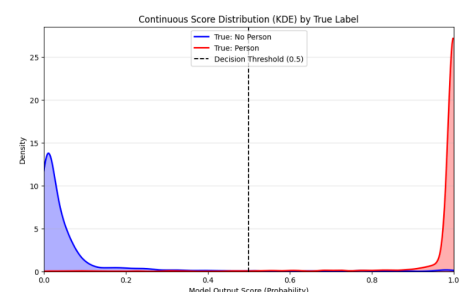
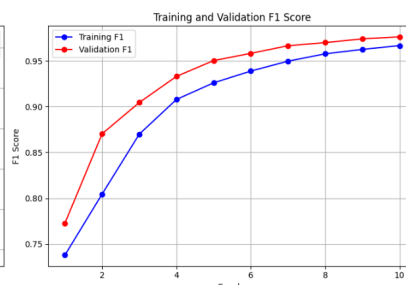


Image 10: Training and validation curves for loss and f1 score

Image 11: Score Distribution per class

6.2 From baseline to the best model

Following one step at the time as described above, the different models were trained and evaluated. The results are shown in Table 1:

Model	Train Loss	Train Acc	Train F1	Val Loss	Val Acc	Val F1
Adap Avg Pool	0.6768	0.5858	0.7388	0.6745	0.5911	0.7430
AdamW	0.2973	0.8771	0.8969	0.2318	0.9047	0.9232
Logits	0.2880	0.8843	0.9017	0.2230	0.9082	0.9236
Better ODA	0.2307	0.9082	0.9222	0.1979	0.9235	0.9347
BatchNorm (Final)	0.1771	0.9296	0.9385	0.1380	0.9394	0.9470

Table 1: Results of the training of the different configurations

As expected, the metrics in the first model drop significantly to lower values. Although it is a much worse model than the baseline, we decided to continue with this one and try to improve it, as it was a different design. In the table one can clearly see the additive contribution of each component. The most impactful one was changing the optimizer, which brought back the model performance to similar levels to the baseline. After all the improvements made, the model is capable of achieving comparable results to the baseline, while having less than 100k parameters.

The final model reaches a 94% accuracy rate and a f1 score of 0.95 in the validation set. The curves of the training are smooth and do not show signs of overfitting, see Image 12.

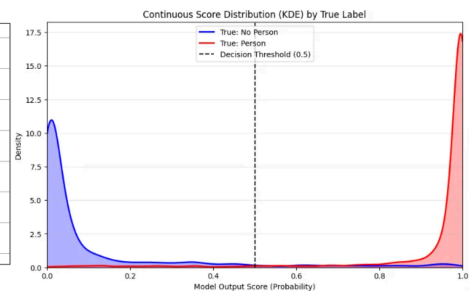
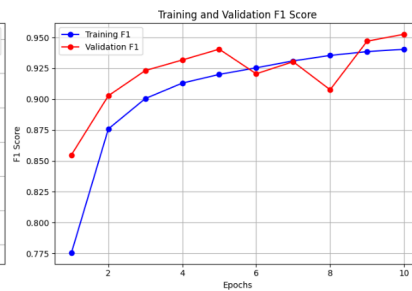


Image 12: Training and validation curves for loss and f1 score

Image 13: Score Distribution per class

Regarding the distribution of the scores of the validation set, the ones produced by this model are similar to the ones produced by the baseline model. Both classes are clearly separated and again, moving the threshold would not change the results. With the current threshold (0,5) the model shows 193 FP and 163 FN. In an alarm model like this one, it is important to minimize the number of FN. As this was the best model in terms of the efficiency - performance trade-off, the model was evaluated on the test set.

6.3 Best Model test evaluation

The results the model gave in the test set are: 81% accuracy and f1 score of 0,83. These results are lower than the ones obtained in the train and validation splits. The difference can be explained by the validation set being too optimistic. As both train and validation contain similar images the results obtained in the validation will be very high.

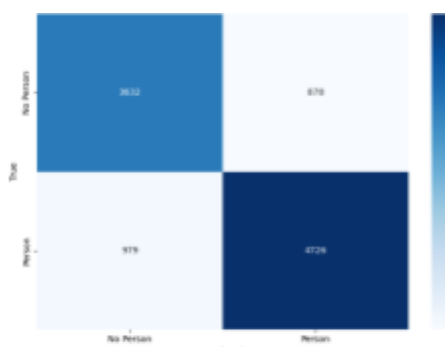


Image 14: Confusion matrix in LLVIP test split

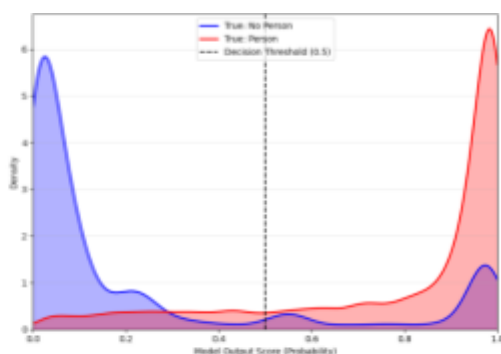


Image 15: Score distribution by class in LLVIP test

Analyzing the distribution of the scores, we can see that they are not perfectly separated and that some cases have ambiguous scores, meaning that the model is undecided in some images. In this case, moving the threshold would have an impact on the results. In a use case like this it is interesting to have more false alarms and not miss true positives. From Image 15, values in the range of 0.3 to 0.4 would be nice fits, not increasing the False Positives but reducing the amount of False Negatives.

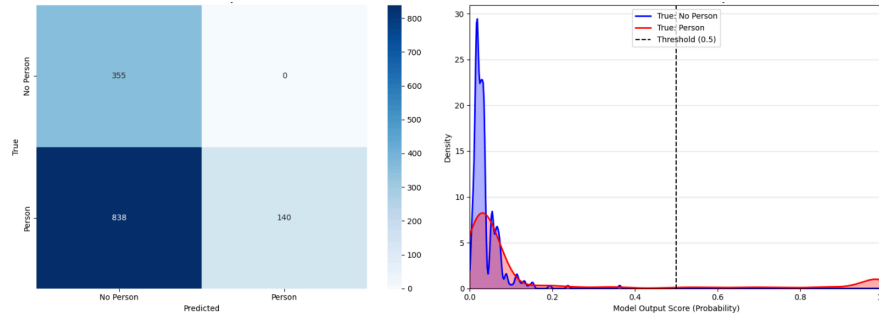


Image 16: Confusion matrix in My Dataset test split Image 17: Score distribution by class in My Dataset test

When evaluating the best model in our handmade dataset the metrics changed drastically. The accuracy is only 37% (less than a random model) and f1 score of 0,25. The confusion matrix and the distribution of scores (Image 16 and 17) clearly show the model is not learning anything useful to classify these images. We conclude that the domain shift between both datasets is too much for the model, and cannot correctly interpret the images, and is not able to detect people because it expects them to appear as bright zones in the image.

6.4 Transfer learning using YOLOv8

The transfer learning model was trained with 933 images from My Dataset with no data augmentation techniques. In the curves of Image 18 representing the training process, we observe a similar case to the ones seen before, where the validation outperforms the training. Here the duplicates are not that common, some background images are inherently similar but different angles and light conditions were applied. The final validation accuracy was 91.00% and a f1 score of 0,94.

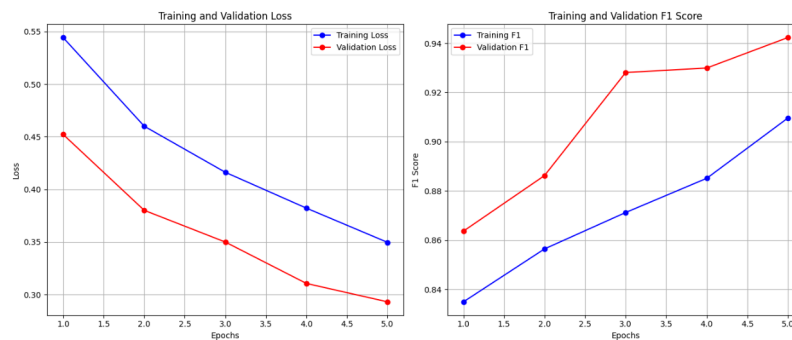


Image 18: Training and validation curves for loss and f1 score

The results are promising so this model was also evaluated on the test set. The results it gave are solid and comparable to the ones obtained in the validation set. The accuracy in the test set is 88.5% and the f1 score is 0,92. To visualize better how it performs the confusion matrix and the score distribution are shown in Image 19 and 20. Here the distribution of scores is not as radically separated as before, so changing the threshold would make a huge impact. The perfect choice is to take as much red area as possible without taking much blue area (that will be new cases of false positives). Changing the threshold here is a delicate decision as the impact is significant. Good values would be in the range from 0.4 to 0.5.

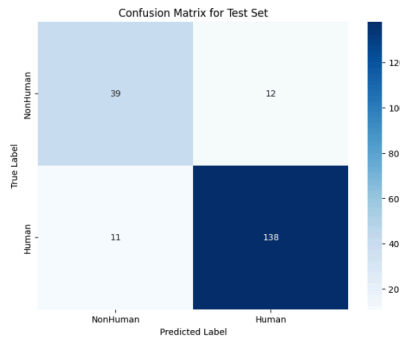


Image 19: Confusion matrix on My Dataset test

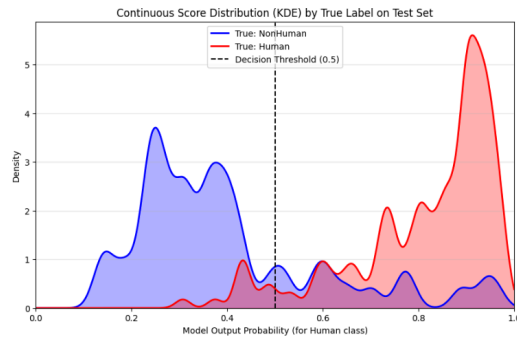


Image 20: Score distribution per class in My Dataset

7. COMPARISON AGAINST BASELINES

7.1 Baseline vs Best Model

The baseline model achieves very good results on the training and validation sets during the training process, remark that these results are over optimistic. After all the incremental updates done to the simplified architecture, the optimized model is able to reach similar performance metrics to those of the baseline model. However, the best model has substantially fewer parameters than the baseline model, 6.5M from the baseline model versus less than 100k of the best model. This makes the Best model clearly better than the baseline for this use case because the evaluation of images can be made faster, this is an important feature of a model that has to be deployed in real time applications.

7.2 CNN models vs Transfer Learning model

Both models were evaluated on the test of My Dataset. In this evaluation the Best Model performs worse than making random assignments while the Transfer learning model performs very well. Even though this difference of results, the comparison is not fair as the transfer learning model is trained on data of the same domain. What can be attributed as an advantage is the capacity that the transfer learning model has to be trained with a small amount of data and give good results, something that would be impossible to achieve with the baseline or the Best Model used with the LLVIP dataset.

8. DISCUSSION OF LIMITATIONS AND OBSERVATIONS

8.1 Domain shift

The main challenge found during this work is the domain shift between the LLVIP dataset and our handmade My Dataset. The fact that one is composed by actually infrared thermal images and the other is just regular black and white from a regular security camera makes them very different. LLVIP images capture human bodies as bright thermal signatures against cooler backgrounds, making the classification more easy. The pool dataset, on the other hand, has low-contrast images where humans appear as grey shapes in most cases. The model that is trained only in thermal infrared images cannot recognize humans in these photographs. That is why the transfer learning model was developed, to be able to perform the same task in a case like the one presented in My Dataset. However it would be nice to have one model that generalizes good enough to be deployed in both situations.

8.2 Difficult cases

In both datasets there are images that are very difficult to classify, even for us. These images are images where the person is behind something, in a motorcycle or in the case of our dataset, has very low contrast conditions. These cases are inherently ambiguous and represent a hard limit on achievable accuracy regardless of model sophistication.

8.3 Small and imbalanced dataset and duplicates

The dataset made only contains 1333 images, and there is an imbalance between both classes (73% of images with people and 27% without). The small size of the dataset only allows to do transfer learning and makes evaluation more difficult as the statistics are less reliable. Regarding the LLVIP dataset, due to the division technique applied, there are images that are exactly or near duplicates of other images. The fact that both train and validation were done randomly from the same directory makes them contain similar images, making the validation metrics appear as extremely good and differ from the metrics gotten when evaluating the model in the test split, which was done with another directory containing images from other completely different cameras.

8.4 Normalization mismatch

The YOLOv8 backbone was pretrained on RGB images from large-scale datasets, where almost all of them are daylight images. The normalization applied to the three duplicated channels are based on ImageNet normalization statistics [4], which might not be a perfect fit for our images, because this mismatch means the backbone receives inputs that lie outside the distribution it was optimized for, limiting feature extraction quality. Other options can be tested, for example the mean and standard deviation of our own dataset also duplicated 3 times.

9. CONCLUSIONS AND FUTURE WORK

9.1 Conclusions

The results and all the process of making this project gave some key takeaways and findings:

- The two models obtained perform very well in their designated tasks. The Best Model trained on LLVIP can achieve 93.9% validation accuracy and 0.9470 F1 score. Transfer learning using a YOLOv8 backbone achieved 90% accuracy and 0.94 F1 score.
- Batch normalisation, AdamW optimisation, and logits-based loss (BCEWithLogitsLoss) are the three most impactful design choices for this binary classification task. They bring stabilization and performance improvements.
- Transfer learning is able to achieve good results with not much training data and epochs.
- Adapting a detection model that is able to recognize all types of objects to a human presence classification task is feasible and gives good results.
- Image diversity in the training set is critical: a model trained on a single type of infrared scene may not generalise to other cameras or environments
- The choice of architecture matters less than proper training procedures: a lightweight custom CNN with ~100K parameters, trained with appropriate regularisation, matches a much larger pretrained model on this task.

9.2 Future Improvements

Here we present the next improvements and tests that could be performed to enhance this project:

- Expand MyDataset to have more variety of images and compensate the imbalance between classes
- Explore more how domain adaptation could be done. For example, exploring domain randomization augmentations during training to bridge the infrared and normal images gap.
- Investigate models specifically trained on infrared datasets as backbones for transfer learning.
- Do a real-time deployment, integrate the final model in an actual security camera with an automatic notification system.
- Move to an object detection problem formulation, enabling more precise alarms and counting how many people are in the image.

10. REFERENCES

- [1] Ultralytics. (2023, noviembre 12). Explore Ultralytics YOLOv8. Ultralytics.com.
<https://docs.ultralytics.com/models/yolov8/>
- [2] Ultralytics. (2026, enero 29). Backbone. Ultralytics.
<https://www.ultralytics.com/glossary/backbone>
- [3] Lyu, C., Zhang, W., Huang, H., Zhou, Y., Wang, Y., Liu, Y., Zhang, S., & Chen, K. (2022). RTMDet: An empirical study of designing real-time object detectors. En arXiv [cs.CV].
<https://doi.org/10.48550/arXiv.2212.07784>
- [4] torchvision.models — Torchvision master documentation. (s/f). Pytorch.org.
<https://docs.pytorch.org/vision/0.9/models.html>